18
Nearby Friends

Location sharing is a fun way to stay connected with friends and family. In this chapter, we design a location based social network called Nearby Friends. We focus on the backend design and discuss how to scale the system to support 10 million users. We also discuss how to make the system highly available, consistent, and fast.

Step 1 - Understand the Problem and Establish Design Scope

Figure 18.1: Nearby Friends

Location based services (LBS) provide a wide range of services, such as finding the closest restaurants, hotels, movie theaters, or any other points of interest based on your current location. In this chapter, we are asked to design a location based service that shows mobile users nearby friends. The service is optional for users who want to share their location with their friends.

Step 1 - Understand the Problem and Establish Design Scope

Let's start with a few clarifying questions.

Candidate: Is the distance between two users or two points of interest important?

Interviewer: Yes, it is important. We want to calculate the distance between two users. It can be another factor to consider when we recommend nearby points of interest.

Candidate: How many users does the app have?

Interviewer: Assume we have 10 million users. We also assume it is a fast growing startup that anticipates to scale to many users in the future.

Candidate: How many friends can a user have?

Interviewer: Assume everybody can have up to 5000 friends.

Candidate: How often do users update their locations?

Interviewer: Every 30 seconds.

Candidate: Do we need to store location history?

Interviewer: Yes, it can be stored for 1 year. This data can be used for machine learning.

Candidate: Do we need to store user's friend list and their locations on our server?

Interviewer: Yes, we need to store a friend's list on our server. We also need to store their last known location.

Candidate: Do we need to encrypt the data?

Interviewer: Yes, it is important to encrypt the location data.

Candidate: Should we focus on the quality of the following system?

Interviewer: Yes, it is important to make it reliable, scalable, and accurate.

Functional requirements

- A user should see their nearby friends on their mobile app. When the user's friends are more than a certain distance apart, it is not important to show that they are nearby.
- Nearby friends data should be accurate.

Non-functional requirements

- Low latency for receiving nearby friends' locations.
- Reliability is important for overall feature to work.
- Eventual consistency. The nearby friends' location data arriving a little bit late should not impacting user's ability to use the service.

Back of the envelope estimation

We use some back of the envelope calculations to determine the scalability and bandwidth we need for our design. Let's start with a few assumptions.

- The distance between two users is less than 5 miles.
- On average, there are 10 friends per user, and no user has more than 5000 friends.
- The location refresh interval is 30 seconds. On average, 1 million users are active at the same time.
- On average, 100 active users join a nearby friend's group every day.
- Location data is stored for 8 hours. It is not stored permanently.
- Location data is not encrypted. We need to encrypt the data in transit and at rest.
- We need to store a user's friend list and their last known location on our server.

Location QPS

- Concurrent location updates per second = 1 million / 30 seconds = 33,000
- Peak location updates per second = 33,000 * 5 = 165,000

Bandwidth

- Incoming and outgoing traffic = Concurrent location updates per second * number of friends per user = 33,000 * 10 = 330,000 per second

Storage

- Location history = 1 million * 8 hours * 3600 seconds / 30 seconds = 1 billion

Step 2 - Propose High Level Design and Get Buy In

We need to support the following functionalities:

- Receive location updates.
- Store location history.
- Send nearby friends' locations to their friends.

The system is a combination of a client-server and a peer-to-peer system. We use a client-server system to receive and send location updates and to store location data. We use a peer-to-peer system to send location data to friends who are nearby. The peer-to-peer system is a real-time system and we use a client-server system for all other business logic.

High level design

Client
Client
Client